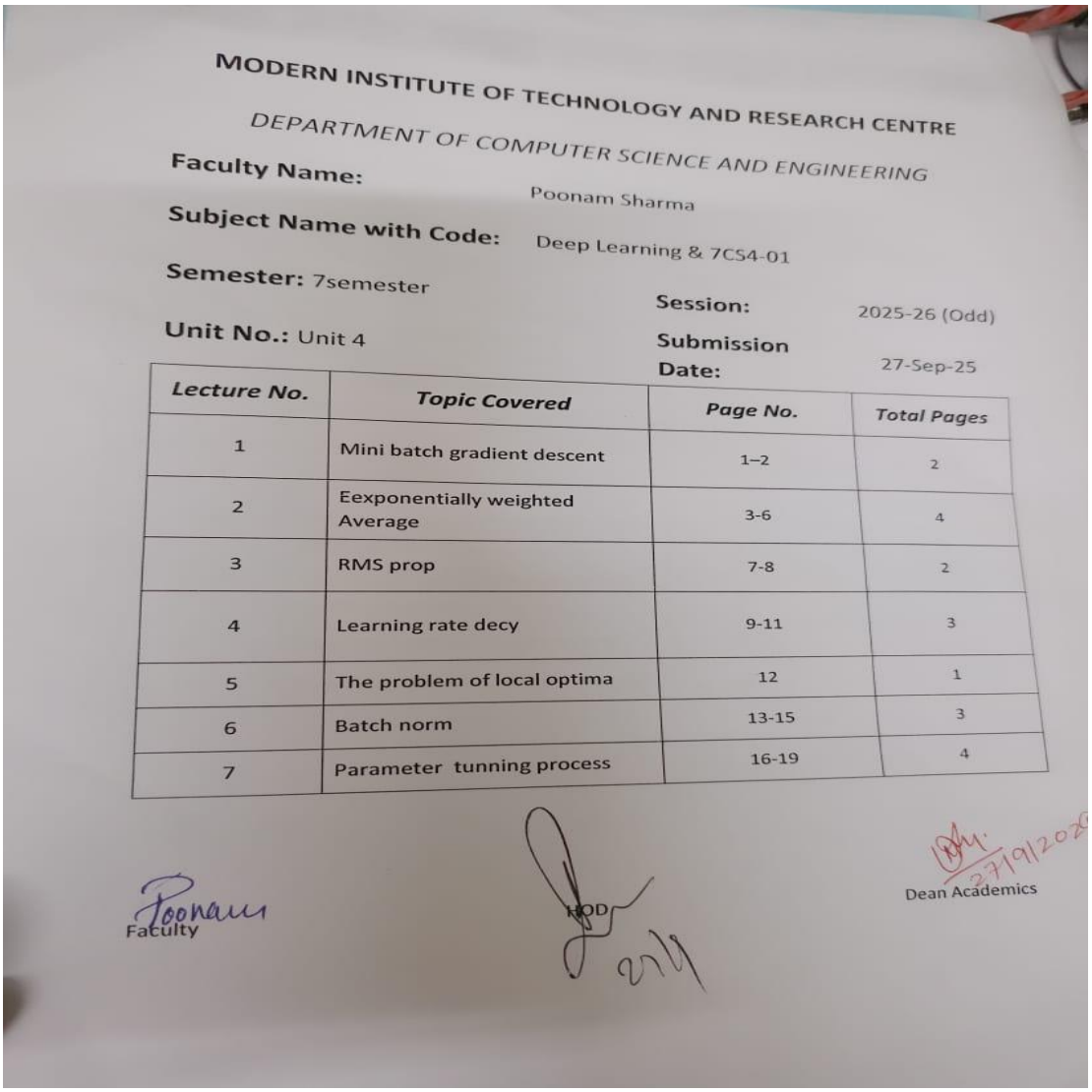


MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name: Poonam Sharma

Subject Name with Code: Deep Learning & 7CS4-01

Semester: 7semester **Session:** 2025-26 (Odd)

Unit No.: Unit 4 **Submission Date:** 27-Sep-25

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	Mini batch gradient descent	1–2	2
2	Eexponentially weighted Average	3-6	4
3	RMS prop	7-8	2
4	Learning rate decy	9-11	3
5	The problem of local optima	12	1
6	Batch norm	13-15	3
7	Parameter tunning process	16-19	4

Poonam
Faculty

HOD

27/9/2025
Dean Academics

UNIT-IV
Optimization Algorithms
Lecture No.1

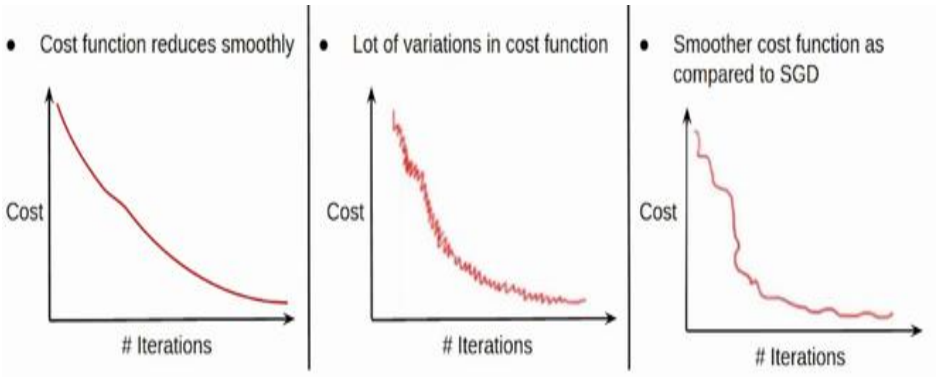
Mini-batch gradient descent

In machine learning, gradient descent is an optimization technique used for computing the model parameters (coefficients and bias) for algorithms like linear regression, logistic regression, neural networks, etc. In this technique, we repeatedly iterate through the training set and update the model parameters in accordance with the gradient of the error with respect to the training set. Depending on the number of training examples considered in updating the model parameters, we have 3-types of gradient descents:

1. **Batch Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to the entire training set.
2. **Stochastic Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to a single training example.
3. **Mini-Batch Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to a subset of the training set.

Batch Gradient Descent	Stochastic Gradient Descent	Mini-Batch Gradient Descent
Since the entire training data is considered before taking a step in the direction of gradient, therefore it takes a lot of time for making a single update.	Since only a single training example is considered before taking a step in the direction of gradient, we are forced to loop over the training set and thus cannot exploit the speed associated with vectorizing the code.	Since a subset of training examples is considered, it can make quick updates in the model parameters and can also exploit the speed associated with vectorizing the code.
It makes smooth updates in the model parameters	It makes very noisy updates in the parameters	Depending upon the batch size, the updates can be made less noisy – greater the batch size less noisy is the update

Thus, mini-batch gradient descent makes a compromise between the speedy convergence and the noise associated with gradient update which makes it a more flexible and robust algorithm.



Convergence in BGD, SGD & MBGD

Mini-Batch Gradient Descent: Algorithm-

Let theta = model parameters and max_iters = number of epochs. for itr = 1, 2, 3, …, max_iters: for mini_batch (X_mini, y_mini):

- *Forward Pass on the batch X_mini:*
  - *Make predictions on the mini-batch*
  - *Compute error in predictions (J(theta)) with the current values of the parameters*
- *Backward Pass:*
  - *Compute gradient(theta) = partial derivative of J(theta) w.r.t. theta*
- *Update parameters:*
  - *theta = theta – learning_rate*gradient(theta)*

References

1. https://www.geeksforgeeks.org/mini-batch-gradient-descent-in-deep-learning/

UNIT-IV(Optimization Algorithms)
Exponentially weighted Averages
Lecture No.2

This algorithm is one of the most important algorithms currently in usage. From financial time series, signal processing to neural networks, it is being used quite extensively. Basically, any data that is in a **sequence**. **This algorithm has been mostly used to reduce the noisy time-series data.** It's also called "**smoothing**" the data.

The way we achieve this is by essentially **weighing the number of observations and using their average.** This is called as **Moving Average.** Example: Temperature θt over days, calculate the moving averages.

Math Notation "sub: subscript", "sup: superscript"

Vt: Moving average value at day 't'

Vsub00 = 0

Vsub1=0.9Vsub0+0.1θsub1

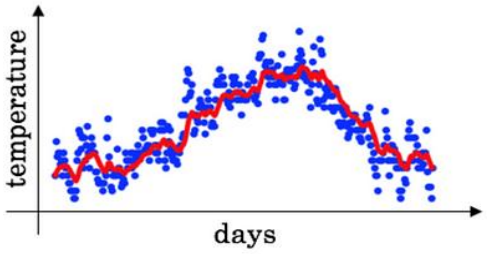
Vsub1=0.9Vsub1+0.1θsub2

..

$V_t = 0.9V_{t-1} + 0.1\theta_t$

if $\beta = 0.9$

$V_t = \beta V_{t-1} + (1-\beta)\theta_t$

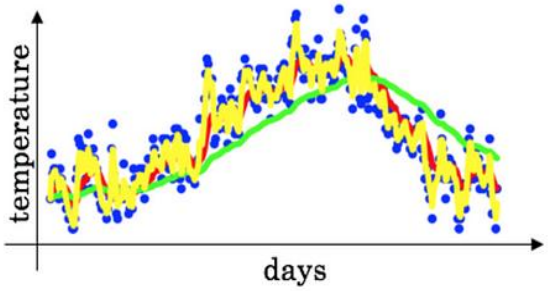


The above equation would give the moving average line in Red over the data points in Blue.

What does and means

Vt: averaging over **1/1−β** days (approx)

For ex. , For **β** =0.9, **1/1−β~=10** ;
β = 0.9 averages over 10 days (smooth curve: Red Line)
For **β** =0.98, ~=50 ;
β = 0.98 averages over 50 days (smoother curve: Green Line) — Not very accurate representation
For **β** =0.5, ~=2 ;
β = 0.5 averages over 2 days (alterations: Yellow Line) — Much more noisy
The right value of is **β** calculated using HyperParameter Tuning

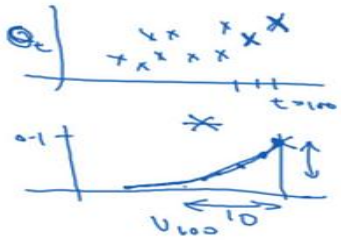


What is Exponentially Weighted Average Actually Doing?

$$Vt=\beta Vt-1+(1-\beta)\theta t$$

Going backwards from V100,
Vsub100 = 0.1θsub100 + 0.9Vsub99
Vsub99 = 0.1θsub99 + 0.9Vsub98
Substituting Vsub99,
Vsub100 = 0.1θsub100 + 0.9(0.1θsub99 + 0.9Vsub98)
or, Vsub100 = 0.1θsub100 + 0.9(0.1θsub99 + 0.9(0.1θsub98 + 0.9Vsub97))

Generalizing,
Vsub100 = 0.1θsub100 + 0.1∗0.9∗θsub99 + 0.1∗$(0.9)^2$θsub98 + 0.1∗$(0.9)^3$θsub97 + 0.1∗$(0.9)^4$θsub96+…..
Vsub100 is basically an element-wise computation of two metrics/functions — one an exponential decay function containing diminishing values (0.9, 0.9^{22}, $0.^{93}$, ……and another with all the elements of θt.



Also, if $\beta = 0.9$, the weight decays to about a third by 10th iteration.

Proof: $(1-\epsilon)^1/\epsilon = 1/\epsilon$
$\epsilon = 1-\beta$

of $\beta = 0.9$, $\epsilon = 1-\beta = 0.1$

$(1-\epsilon)^1/\epsilon = 0.9^{10} = 0.35 = 1/\epsilon$

Interpretation: It takes about 10 days for height to decay to 1/3rd

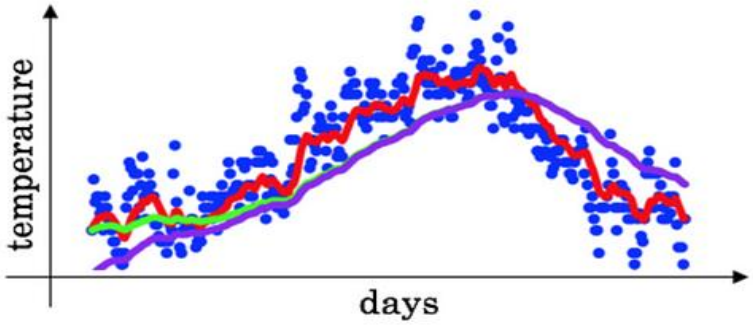
If $\beta = 0.98$, $\epsilon = 0.02$, $1/\epsilon = 50$;
It takes approx 50 days for height to decay to 1/3rd

Implementing Exponentially Weighted Average

Vsubθ: v is computing exponentially weighted average of parameter θ.
day 0: Vsubθ = 0 day 1: Vsubθ=βv+(1−β)θsub1
day 2: Vsubθ = βv+(1−β)θsub2
…
Algorithms: Vsubθ =0 Repeat: {
Get next θt
Vθ := βVsubθ+(1−β)θt
}
Single line implementation for fast and efficient calculation of exponentially weighted moving average.

Bias Correction in Exponentially Weighted Moving Average

Making EWMA more accurate — Since the curve starts from 0, there are not many values to average on in the initial days. Thus, the curve is lower than the correct value initially and then moves in line with expected values.
Figure: The ideal curve should be the GREEN one, but it starts as the PURPLE curve since the values initially are zero



Example: Starting from t=0 and moving forward,

Vsub0 = 0Vsub1 = 0.98Vsub 0+0.02θsub1 = 0.020θsub1

Vsub2 = 0.98Vsub1 + 0.02θsub2 = 0.0196θsub1+0.02θsub2

The initial values of Vt will be very low which need to be compensated.

Make **Vt = Vt/1−βsupt**

for t=2, $1-\beta^{supt} = 1-0.9^{82} = 0.0396$ (Bias Correction Factor)

Vsub2 = V2/0.0396 = 0.0196θsub1 + 0.02θsub2 / 0.0396

When t is large, 1/1−βsupt =1, hence bias correction factor has no effect when t is sufficiently large.

References

1. https://www.investopedia.com/articles/07/ewma.asp

UNIT-IV(Optimization Algorithms)
RMS Prop
Lecture No.3

Gradient descent is an optimization algorithm used to find the set of parameters (coefficients) of a function that minimizes a cost function. This method iteratively adjusts the coefficients of the function until the cost reaches the local, or global, minimum.

Gradient descent works by calculating the partial derivatives of the cost function with respect to each of the coefficients. The algorithm then makes small adjustments to the coefficients in order to reduce the cost until it reaches a minimum. The direction of the adjustments is determined by the negative of the gradient of the cost function. Optimizers are methods or algorithms that reduce a loss (an error) by adjusting various parameters and weights, minimizing the loss function, and thereby improving model accuracy and speed. One such optimization technique is RMSprop.

RMSProp (Root Mean Squared Propagation) is an adaptive learning rate optimization algorithm.It is an extension of the popular Adaptive Gradient Algorithm and is designed to dramatically reduce the amount of computational effort used in training neural networks. This algorithm works by exponentially decaying the learning rate every time the squared gradient is less than a certain threshold. This helps reduce the learning rate more quickly when the gradients become small. In this way, RMSProp is able to smoothly adjust the learning rate for each of the parameters in the network, providing a better performance than regular Gradient Descent alone.

The RMSprop algorithm utilizes exponentially weighted moving averages of squared gradients to update the parameters. Here is the mathematical equation for RMSprop:

1. Initialize parameters:
   - Learning rate: α
   - Exponential decay rate for averaging: γ
   - Small constant for numerical stability: ε
   - Initial parameter values: θ
2. Initialize accumulated gradients (Exponentially weighted average):
   - Accumulated squared gradient for each parameter: $Et = 0$
3. Repeat until convergence or maximum iterations:
   - Compute the gradient of the objective function with respect to the parameters:

$$g_t = \nabla_\theta J(\theta_t)$$

- Update the exponentially weighted average of the squared gradients:

$$E_t = \gamma E_{t-1} + (1-\gamma) g_t^2$$

- Update the parameters:

$$\theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{E_t + \epsilon}}$$

where,

- *gt* is the gradient of the loss function with respect to the parameters at time *t*
- γ is a decay factor
- *Et* is the exponentially weighted average of the squared gradients
- α is the learning rate
- ϵ is a small constant to prevent division by zero.

This process is repeated for each parameter in the optimization problem, and it helps adjust the learning rate for each parameter based on the historical gradients. The exponential moving average allows the algorithm to give more importance to recent gradients and dampen the effect of older gradients, providing stability during optimization.

References

1. https://www.geeksforgeeks.org/rmsprop-optimizer-in-deep-learning/

UNIT-IV(Optimization Algorithms)
Learning Rate Decay
Lecture No.4

Imagine you're looking for a coin you dropped in a big room. At first, you take big steps, covering a lot of ground quickly. But as you get closer to the coin, you take tinier steps to look more precisely. This is similar to how learning rate decay works in machine learning.

In training a machine learning model, the "learning rate" decides how much we adjust the model in response to the error it made. Start with a high learning rate, and the model might learn quickly, but it can overshoot and miss the best solution. Start too low, and it might be too slow or get stuck. So, instead of keeping the learning rate constant, we gradually reduce it. This method is called "learning rate decay." We start off taking big steps (high learning rate) when we're far from the best solution. But as we get closer, we reduce the learning rate, taking smaller steps, and ensuring we don't miss the optimal solution. This approach helps the model train faster and more accurately.

There are various ways to reduce the learning rate: some reduce it gradually over time, while others drop it sharply after a set number of training rounds. The key is to find a balance that lets the model learn efficiently without missing the best possible solution.

Learning Rate Decay

Learning rate decay is a technique used in machine learning models, especially deep neural networks. It is sometimes referred to as learning rate scheduling or learning rate annealing. Throughout the training phase, it entails gradually lowering the learning rate. Learning rate decay is used to gradually adjust the learning rate, usually by lowering it, to facilitate the optimization algorithm's more rapid convergence to a better solution. This method tackles problems that are frequently linked to a fixed learning rate, such as oscillations and sluggish convergence.

Learning rate decay can be accomplished by a variety of techniques, such as step decay, exponential decay, and 1/t decay. Degradation strategy selection is based on the particular challenge and architecture. When training deep learning models, learning rate decay is a crucial hyperparameter that, when used properly, can result in faster training, better convergence, and increased model performance.

How Learning Rate Decay works?

Learning rate decay is like driving a car towards a parking spot. At first, you drive fast to reach the spot quickly. As you get closer, you slow down to park accurately. In machine learning, the learning rate determines how much the model changes based on the mistakes it makes. If it's too high, the model might miss the best fit; too low, and it's too slow. Learning rate decay starts with a higher learning rate, letting the model learn fast. As training progresses, the rate gradually decreases, making the model adjustments more precise. This ensures the model finds a good solution efficiently. Different methods reduce the rate in various ways, either stepwise or smoothly, to optimize the training process.

Mathematical representation of Learning rate decay

A basic learning rate decay plan can be mathematically represented as follows:

Assume that the starting learning rate is η_0 and that the learning rate at epoch t is η_t.A typical decay schedule for learning rates is based on a constant decay rate α, where $\alpha \epsilon (0, 1)$, applied at regular intervals (e.g., every n epochs):

$$\eta_t = \frac{\eta_0}{1+\alpha \cdot t}$$

Where,

- η_t is the learning rate at epoch t.
- η_0 is the initial learning rate at the start of training.
- α is the fixed decay rate, typically a small positive value, such as 0.1 or 0.01.
- t is the current epoch during training.
- The learning rate η_t decreases as t increases, leading to smaller step size as training progresses.

The learning rate is decreased by a percentage of its previous value at each epoch in this formula, which depicts a basic learning rate decay schedule. A timetable like this facilitates the optimization process by enabling the model to converge more quickly at first, then fine-tuning in smaller increments as it gets closer to a local minimum.

Basic decay schedules

In order to enhance the convergence of machine learning models, learning rate decay schedules are utilized to gradually lower the learning rate during training. Here are a few simple schedules for learning rate decay:

- **Step Decay**: In step decay, after a predetermined number of training epochs, the learning rate is decreased by a specified factor (decay rate). The mathematical formula for step decay is:

$$lr = lr_{initial} * drop\ rate^{\frac{epoch}{step\ size}}$$

- **Exponential Decay**: The learning rate is progressively decreased over time by exponential decay. At each epoch, a factor is used to adjust the learning rate. The mathematical formula for Exponential decay is:

$$lr = lr_{initial} * e^{-decay\ rate * epoch}$$

- **Inverse Time Decay**: A factor inversely proportional to the number of epochs is used to reduce the learning rate through inverse decay. The mathematical formula for Inverse Time decay is:

$$lr = lr_{initial} * \frac{1}{1+decay*epoch}$$

- **Polynomial Decay**: When a polynomial function, usually a power of the epoch number, is followed, polynomial decay lowers the learning rate.The mathematical formula for Polynomial decay is:

$$lr = lr_{initial} * \left(1 - \frac{epoch}{max\ epoch}\right)^{power}$$

In simple words, these schedules adjust the learning rate during training. They help in starting with big steps and taking smaller steps as we get closer to the best solution, ensuring efficiency and precision.

Steps needed to implement Learning Rate Decay

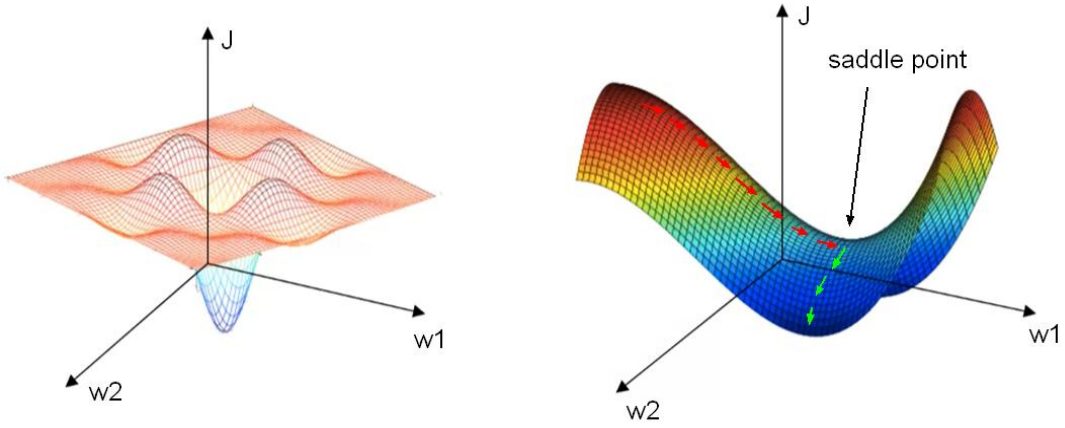
- **Set Initial Learning Rate**: Start by establishing a base learning rate. It shouldn't be too high to cause drastic updates, nor too low to stall the learning process.
- **Choose a Decay Method**: Common methods include exponential decay, step decay, or inverse time decay. The choice depends on your specific machine learning problem.
- **Implement the Decay**: Apply the chosen decay method after a set number of epochs, or based on the performance of the model.
- **Monitor and Adjust**: Keep an eye on the model's performance. If it's not improving, you might need to adjust the decay rate or the method.

References

1. https://www.geeksforgeeks.org/learning-rate-decay/

UNIT-IV(Optimization Algorithms)
The Problem of Local Optima
Lecture No.5

Looking at the figure here below on the left, it is quite common to get stuck to a local minimum rather than find the way to a global optimum. In reality, most of the points of zero gradient are not local optima like in the figure on the left, but instead most points of zero gradient are in a so called **saddle points** as pictured on the right figure.



The **saddle point** has derivatives equal to zero and we have the **Problem of Plateaus** which is a region where the derivative is close to zero foor a long time. The **Plateau** can really slow down the learnind. The plateau is represented by the red arrow on the right figure. In fact, the process takes long time before realize to follow the green path in order to reach the minimum.This is the scenario where more sophisticated observation algorithms, such as AdaM, can actually speed up the learning-rate.

References

1. https://www.geeksforgeeks.org/local-optima-and-influence-of-effectiveness-of-local-search-algorithms/

UNIT-IV(Optimization Algorithms)
Batch Norm
Lecture No.6

Batch normalization is a deep learning approach that has been shown to **significantly improve the efficiency and reliability of neural network models**. It is particularly useful for training very deep networks, as it can help to reduce the internal covariate shift that can occur during training.

Batch normalization is a supervised learning method for normalizing the interlayer outputs of a neural network. As a result, the next layer receives a "reset" of the output distribution from the preceding layer, allowing it to analyze the data more effectively.
The term "internal covariate shift" is used to describe the effect that updating the parameters of the layers above it has on the distribution of inputs to the current layer during deep learning training. This can make the optimization process more difficult and can slow down the convergence of the model.

Since normalization guarantees that no activation value is too high or too low, and since it enables each layer to learn independently from the others, this strategy leads to quicker learning rates. By standardizing inputs, the "dropout" rate (the amount of information lost between processing stages) may be decreased. That ultimately leads to a vast increase in precision across the board.

How does batch normalization work?

Batch normalization is a technique used to improve the performance of a deep learning network by first removing the batch mean and then splitting it by the batch standard deviation.

Stochastic gradient descent is used to rectify this standardization if the loss function is too big, by shifting or scaling the outputs by a parameter, which in turn affects the accuracy of the weights in the following layer.

When applied to a layer, batch normalization multiplies its output by a standard deviation parameter (gamma) and adds a mean parameter (beta) to it as a secondary trainable parameter. Data may be "denormalized" by adjusting just these two weights for each output,

thanks to the synergy between batch normalization and gradient descents. Reduced data loss and improved network stability were the results of adjusting the other relevant weights.

The goal of batch normalization is to stabilize the training process and improve the generalization ability of the model. It can also help to reduce the need for careful initialization of the model's weights and can allow the use of higher learning rates, which can speed up the training process.

It is common practice to apply batch normalization prior to a layer's activation function, and it is commonly used in tandem with other regularization methods like a dropout. It is a widely used technique in modern deep learning and has been shown to be effective in a variety of tasks, including image classification, natural language processing, and machine translation.

Advantages of batch normalization

- **Stabilize the training process**. Batch normalization can help to reduce the internal covariate shift that occurs during training, which can improve the stability of the training process and make it easier to optimize the model.
- **Improves generalization**. By normalizing the activations of a layer, batch normalization can help to reduce overfitting and improve the generalization ability of the model.
- **Reduces the need for careful initialization**. Batch normalization can help reduce the sensitivity of the model to the initial weights, making it easier to train the model.
- **Allows for higher learning rates**. Batch normalization can allow the use of higher learning rates that can speed up the training process.

Batch normalization overfitting

While batch normalization can help to reduce overfitting, it is not a guarantee that a model will not overfit. Overfitting can still occur if the model is too complex for the amount of training data, if there is a lot of noise in the data, or if there are other issues with the training process. It is important to use other regularization techniques like dropout, and to monitor the performance of the model on a validation set during training to ensure that it is not overfitting.

Batch normalization equations

During training, the activations of a layer are normalized for each mini-batch of data using the following equations:

- Mean: mean = 1/m ∑i=1 to m xi

- Variance: variance = 1/m ∑i=1 to m (xi – mean)^2
- Normalized activations: yi = (xi – mean) / sqrt(variance + ε)
- Scaled and shifted activations: zi = γyi + β, where γ and β have learned parameters
  During inference, the activations of a layer are normalized using the mean and variance of the activations calculated during training, rather than using the mean and variance of the mini-batch:
- Normalized activations: yi = (xi – mean) / sqrt(variance + ε)
- Scaled and shifted activations: zi = γyi + β

References

1. https://en.wikipedia.org/wiki/Batch_normalization

UNIT-IV(Optimization Algorithms)
Parameter Tuning Process
Lecture No.7 & Lecture No.8

What are hyperparameters?

In Machine Learning/Deep Learning, a model is represented by its parameters. In contrast, a training process involves selecting the best/optimal hyperparameters that are used by learning algorithms to provide the best result. So, what are these hyperparameters? The answer is, "***Hyperparameters are defined as the parameters that are explicitly defined by the user to control the learning process.***"

Here the prefix "hyper" suggests that the parameters are top-level parameters that are used in controlling the learning process. The value of the Hyperparameter is selected and set by the machine learning engineer before the learning algorithm begins training the model. **Hence, these are external to the model, and their values cannot be changed during the training process**.

Some examples of Hyperparameters in Machine Learning

- The k in kNN or K-Nearest Neighbour algorithm
- Learning rate for training a neural network
- Train-test split ratio
- Batch Size
- Number of Epochs
- Branches in Decision Tree
- Number of clusters in Clustering Algorithm

Difference between Parameter and Hyperparameter

There is always a big confusion between Parameters and hyperparameters or model hyperparameters. So, in order to clear this confusion, let's understand the difference between both of them and how they are related to each other.

Model Parameters

Model parameters are configuration variables that are internal to the model, and a model learns them on its own. For example**, W Weights or Coefficients of independent variables in the Linear regression model**. or **Weights or Coefficients of independent variables in SVM, weight, and biases of a neural network, cluster centroid in clustering.** Some key points for model parameters are as follows:

- They are used by the model for making predictions.
- They are learned by the model from the data itself
- These are usually not set manually.
- These are the part of the model and key to a machine learning Algorithm.

Model Hyperparameters

Hyperparameters are those parameters that are explicitly defined by the user to control the learning process. Some key points for model parameters are as follows:

- These are usually defined manually by the machine learning engineer.
- One cannot know the exact best value for hyperparameters for the given problem. The best value can be determined either by the rule of thumb or by trial and error.
- Some examples of Hyperparameters are **the learning rate for training a neural network, K in the KNN algorithm,**

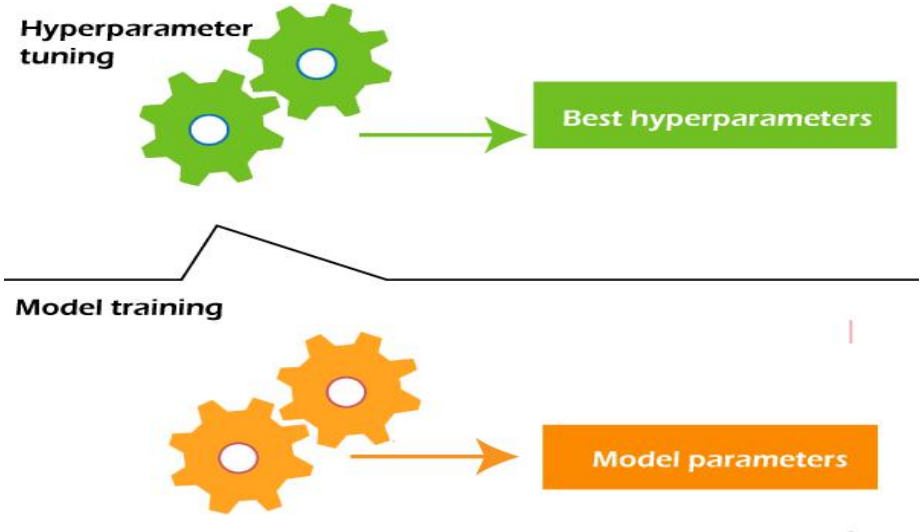
Categories of Hyperparameters

Broadly hyperparameters can be divided into two categories, which are given below:

- **Hyperparameter for Optimization**
- **Hyperparameter for Specific Models**

Hyperparameter for Optimization

The process of selecting the best hyperparameters to use is known as hyperparameter tuning, and the tuning process is also known as hyperparameter optimization. Optimization parameters are used for optimizing the model.



Some of the **popular optimization parameters** are given below:

- **Learning Rate:** The learning rate is the hyperparameter in optimization algorithms that controls how much the model needs to change in response to the estimated error for each time when the model's weights are updated. It is one of the crucial parameters while building a neural network, and also it determines the frequency of cross-checking with model parameters. Selecting the optimized learning rate is a challenging task because if the learning rate is very less, then it may slow down the training process. On the other hand, if the learning rate is too large, then it may not optimize the model properly.

- **Batch Size:** To enhance the speed of the learning process, the training set is divided into different subsets, which are known as a batch. **Number of Epochs:** An epoch can be defined as the complete cycle for training the machine learning model. Epoch represents an iterative learning process. The number of epochs varies from model to model, and various models are created with more than one epoch. To determine the right number of epochs, a validation error is taken into account. The number of epochs is increased until there is a reduction in a validation error. If there is no improvement in reduction error for the consecutive epochs, then it indicates to stop increasing the number of epochs.

Hyperparameter for Specific Models

Hyperparameters that are involved in the structure of the model are known as hyperparameters for specific models. These are given below:

- **A number of Hidden Units:** Hidden units are part of neural networks, which refer to the components comprising the layers of processors between input and output units in a neural network.It is important to specify the number of hidden units hyperparameter for the neural network. It should be between the size of the input layer and the size of the output layer. More specifically, the number of hidden units should be 2/3 of the size of the input layer, plus the size of the output layer. For complex functions, it is necessary to specify the number of hidden units, but it should not overfit the model.

- **Number of Layers:** A neural network is made up of vertically arranged components, which are called layers. There are mainly **input layers, hidden layers, and output**

layers. A 3-layered neural network gives a better performance than a 2-layered network. For a Convolutional Neural network, a greater number of layers make a better model.

References

1. https://www.run.ai/guides/hyperparameter-tuning
2. Deep Learning Step by Step with Python, N D Lewis, 2016